R1D4 Robot User Interface

ECSE 458D1: Capstone Design Project



Group: DP 36

Abonty, Noorjahan - ID: 260953393 (noorjahan.abonty@mail.mcgill.ca)
Bizzarri, Paolo - ID: 261031842 (paolo.bizzarri@mail.mcgill.ca)
Hershkop, Joseph - ID: 260983531 (joseph.hershkop@mail.mcgill.ca)
Singhal, Anjali - ID: 261011655 (anjali.singhal@mail.mcgill.ca)

Project Advisor:

Joseph Vybihal (joseph.vybihal@mcgill.ca)

December 18, 2024

Abstract

The overall project aims to enhance two robotics systems: R1D4, a multifunction robot with a facial interface for user interaction, and a swarm of robots capable of performing coordinated tasks. This report will refer exclusively to the first robotics system, the R1D4. The R1D4 is a robot designed to operate in a hospital setting autonomously to deliver medical supplies between rooms. The current design of the robot lacks any user interface for this robot that a patient or doctor could interact with. Our goal was to create an interface on a tablet that a user could work with in order to schedule and manage deliveries. The first sub goal that we needed to complete is the creation of a communication network between the tablet, the robot and a Telnet server currently used to guide the robot's movements. The second is to create an Android application that allows the user to see what process/states the robot is in and plan schedules. This semester we fully completed the communication network, which can be demonstrated using a test application running on the tablet. We partially completed the Android application. We used an assortment of frameworks and tools to complete these tasks, including React JS, Android Studio, Arduino IDE, an ESP32 with Telnet and an Android tablet.

Acknowledgments

We would like to thank Capstone group 17 - Bilar Mokhtari, Walid Aissa, Jules Delabrousse, Zakaria Benhssein. They helped familiarize us with the current state of the R1D4 hardware.

Table of contents

1. Abbreviations and/or Notations Used in the Report **4**
2. Introduction/Motivation/Objective **4**
3. Background **4**
3.1 Introduction to Project Components
3.2 ESP32 Microcontroller
3.3 Choice of C++
3.4 Telnet Server
3.5 Frontend Frameworks and Tools
3.6 Android USB Host API
3.7 XML Files
4. Requirements and/or Problems **8**
4.1 Requirements
4.2 Constraints
5. Design and Results **8**
5.1 Decisions Made
5.1.1 ESP32 Code Fork
5.1.2 Android App vs Web App
5.1.3 Android tablet Purchase
5.1.4 Frontend Framework
5.2 Results and Progress
5.2.1 Frontend Prototype
5.2.2 Control Flow
5.2.3 ESP32 Serial Communication Code
5.2.4 ESP32 Telnet Communication Code
5.2.5 ESP32 to Tablet Communication
6. Plan for Next Semester **14**
6.1 Remaining Task and Timeline
6.2 Testing Plan
6.3 Testing-Oriented Design Decisions
6.4 Project Objectives
7. Impact on Society and the Environment **15**
7.1 Use of Non-Renewable Resources
7.2 Environmental Benefits
7.3 Safety and Risk
7.4 Benefits to Society
8. Teamwork Report **16**
8.1 Team Collaboration
8.2 Individual Contributions

9. Conclusion **17**
10. References **18**
11. Appendices **18**

1. Abbreviations and/or Notations Used in the Report

UI: User Interface
GUI: Graphical User Interface
ESP32: Espressif32

2. Introduction/Motivation/Objective

Our project focuses on creating an intuitive touchscreen interface to aid the deployment of the R1D4 robot in hospital settings. This robot is designed to autonomously deliver medical supplies, but currently it lacks a system that allows users such as hospital staff to interact with it effectively. The primary goal of the project is to develop a tablet-based interface that enables users to schedule and manage deliveries, monitor the robot's real-time status through the robot's use of animated expressive features.

This main goal requires the implementation of two sub goals. The first is the creation of the frontend Android application described above. This involves creating software that can display facial animations on the Android tablet, and creating the software responsible for managing the delivery schedule. In addition, the tablet needs to be able to communicate to the robot. As of the current state of the project, all of the robot's action commands (move/spin/read from sensors) are being sent through a Telnet server. Therefore, this project requires establishing a reliable communication scheme between the tablet, the robot (which is operated using the ESP32 microcontroller), and a Telnet server.

This project holds significant importance, as the R1D4 robot is designed for deployment in hospital settings, where it will assist staff in performing their tasks more efficiently. Given the diversity of potential users including individuals with no technical background, the interface must be user-friendly, intuitive, and easy to navigate. This ensures that hospital staff can comfortably interact with the robot, assign it tasks, and monitor its operations without requiring extensive training or technical expertise.

In addition to facilitating task assignments and interaction, the interface will also serve as a data management tool. It will enable the logging and tracking of delivery data, such as verifying whether the correct medicines were delivered and in a timely manner, tracking quantities, and maintaining records for future reference. This capability is important because it helps maintain accountability, ensuring compliance with hospital protocols.

Beyond its immediate application in hospital settings, our project demonstrates the broader potential of integrating robotics with human-centered interfaces. Such systems could be adapted for use in other sectors, such as logistics, hospitality, or retail, where efficient task management and data tracking are equally valuable. Our project's emphasis on accessibility and functionality means it has the potential to contribute meaningfully to the adoption of robotics in environments where ease of use and reliability are prioritized.

3. Background

3.1 Introduction to Project Components

Our project involves creating an interface for the R1D4 robot using an ESP32 microcontroller to manage the communication between an Android tablet and the robot. An architectural overview of the system components is presented below in figure 1:

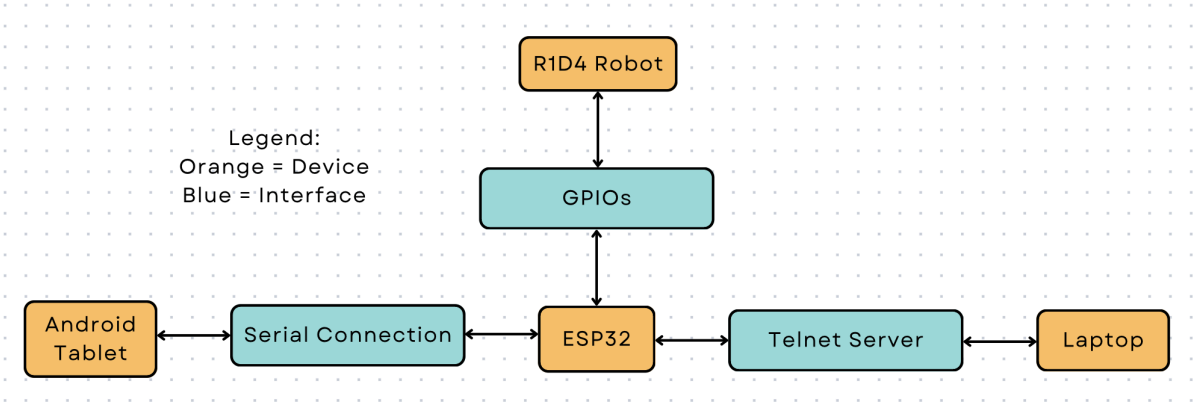


Figure 1: Architectural Overview of System Components

3.2 ESP32 Microcontroller

The ESP32 is a microcontroller known for its wireless capabilities, which build upon the capabilities of a standard Arduino board due to its built-in wireless features. These features enable wireless communication and data exchange capabilities crucial for our project. The ESP32 supports both wired and wireless connections simultaneously. One notable feature is its ability to send commands and receive data over a serial connection through a micro USB interface, acting as an intermediary between the hardware interface of our robot and the Android-based user interface.

In addition, the ESP32 is significantly more powerful than an Arduino Uno R3. It sports a dual-core processor with a clock speed of up to 240 MHz, surpassing the Arduino Uno R3's single-core 16 MHz processor. In terms of memory, the ESP32 offers 520 KB of SRAM, whereas the Arduino Uno R3 is limited to 2 KB. The ESP32 can also handle more complex tasks due to its greater number of GPIO pins (36 compared to the Uno's 14). Overall, the ESP32's higher processing power, ample memory, and integrated wireless features make it a superior choice for this project, where it is important to maintain plenty of room for overhead as the project expands into future terms.

3.3 Choice of C++

To synchronize the ESP32 with the user interface, our team needed to write and test code in either C++ or MicroPython. Using C++ with the ESP32 was the better choice for us over MicroPython. It has superior performance, as the code is compiled directly to machine code, resulting in faster execution and lower resource usage compared to the interpreted nature of MicroPython. Additionally, C++ provides fine-grained control over hardware features, such as timers, interrupts, and peripherals, which is crucial for our application that relies on precise inputs and outputs to and from R1D4. Since this area is relatively new to us, the extensive Arduino community offers a wealth of examples, tutorials, and support, making it easier for us to learn and debug. Finally, the pre-existing code from the previous term had been written in C++. Granted, the code we were given did not work correctly, as it misused the analog write functions of the ESP32. Because of the rationale outlined above, we learned basic C++ programming and set up the Arduino Integrated Development Environment (IDE) to work with the ESP32 board. The Arduino IDE allowed us to write, upload, and test code needed for our application.

3.4 Telnet Server

In order to communicate with the robot from a remote laptop, we are required to set up a server. Telnet is a network protocol used for text-based communication with remote devices or servers. It allows

a user to log in to another computer over a TCP/IP network and execute commands as if they were directly accessing the system. Telnet operates on port 23 by default. It operates on a client-server model, where the Telnet client establishes a connection with a Telnet server to initiate a session. However, Telnet lacks encryption, transmitting data, including sensitive credentials, in plain text, which makes it insecure by modern standards. Despite this limitation, it was widely used historically for remote system administration and troubleshooting tasks. For our needs, however, these limitations are acceptable, and telnet is a straightforward and convenient protocol to use on the ESP32 to get a server up and running.

We also needed to understand network protocols for establishing a Telnet connection. This involved learning about Internet Protocol (IP) addresses and their significance in networking. Because the ESP32 can only connect to 2.4GHz networks, our team needed to learn how to access and configure home router settings to create a dedicated 2.4GHz network to establish a connection with Telnet.

3.5 Frontend Frameworks and Tools

To implement the user interface, we developed an Android application that runs locally on an Android tablet. The project was built using React Native with Expo, which was a new framework that none of the team members had worked with previously, so we had to familiarise ourselves with it. React Native is an open-source JavaScript framework widely utilized for creating mobile applications. It is based on React, a popular library for building front-end user interfaces. React Native offers a variety of pre-built UI components that facilitate rapid development and deployment of applications. By leveraging React JavaScript for the app's interface, React Native ensures improved responsiveness and reduced load times, enhancing the overall user experience [2]. This framework is particularly suited for developing applications with both simple and complex designs due to its reactive UI and component-based architecture.

Although React Native can be used independently, integrating a framework enhances productivity by providing APIs and tools for building production-ready applications. For this reason, Expo, a free and open-source production-grade framework for React Native, was chosen. Expo simplifies the setup and debugging process while offering developer tools that streamline app development. These tools include file-based routing, a comprehensive library of universal components, and a standardized set of native modules [3].

The frontend of the application was tested using an emulator configured with Android Studio. The Android Emulator replicates Android devices on a computer, enabling the testing of applications across various device types and API levels without requiring access to multiple physical devices.

The animation of the faces in the UI is implemented using Lottie, which is a free library that is used to add animations to React Native applications. It works by rendering animations created in Adobe After Effects and exported as JSON files using the Bodymovin plugin. Lottie is available as a package for React Native, typically implemented through the lottie-react-native library. It provides lightweight and high-quality animations, which can even be customized.

3.6 Android USB Host API

One of the essential parts of this project is to build a communication network between the Android tablet and the ESP32. To implement this, we need a solid understanding of how USB communication works as it relates to Android devices. For this project, we utilized the Android USB Host Mode (OTG) API, specifically the version implemented in the GitHub repository **usb-serial-for-Android** as referenced further down the report.

The Android USB Host API enables Android devices to act as USB hosts, allowing them to power and communicate with connected USB devices. When in USB host mode, the Android device supplies power to the USB bus, enumerates connected devices, and facilitates data transfer through defined communication protocols. This final functionality is what we care about most in this case. To leverage this functionality, applications must include the Android.hardware.usb.host feature in their manifest and declare specific filters using <intent-filter> and <meta-data> elements. These filters help identify the USB devices the application wants to communicate with, either by specifying attributes like vendor ID and product ID or by targeting device classes, subclasses, and protocols.

The API revolves around several key classes within the Android.hardware.usb package. The UsbManager class serves as the entry point for USB operations, allowing developers to enumerate and access connected devices. Each connected device is represented by a UsbDevice object, which provides identifying information such as vendor ID, product ID, and device interfaces. Communication is handled through a UsbDeviceConnection, which facilitates data exchange .

In the specific library we utilize for our project, there are a couple additional objects that we utilize. The UsbSerialDriver class acts as an object that contains all the devices connected to the Android tablet through USB connection that can communicate via serial. The UsbSerialPort object represents a specific port in the USB device that we can access. In our case, there is only ever going to be one port but it is important to note the general software architecture. We also use the UsbSerialProber object to grab the available drivers connected to the device [1].

This library also makes it easy to recognize when a USB device has been connected. This is achieved by using an intent filter. For example, by specifying the Android.hardware.usb.action.USB_DEVICE_ATTACHED intent in the broadcast receiver module (a basic Android Studio object that tells us if something is connected to the device physically or otherwise), the system can prompt the user to launch the application when a compatible device is connected, granting immediate permission to interact with the device. This same logic applies to when a device is removed from the tablet.

Before communicating with a USB device, applications must have user permission. If an intent filter is used to detect a device, permission is automatically granted when the user allows the application to handle the device. Otherwise, developers can explicitly request permission by calling the requestPermission() method, which triggers a dialog prompting the user for approval. We have elected to use this function as it made it much easier to debug the system. Once permission is granted, the application can set up a communication channel by locating the appropriate USBSerialPort of the device, then opening a UsbDeviceConnection to exchange data.

3.7 XML Files

In regard to the test application used to demonstrate the communication between the tablet, ESP32 and the telnet server, there is some small background information that is helpful to know. Specifically, how to create an Android app just using the native design functionality in Android Studio. The first step is to understand what an XML file is. Upon startup, an Android Studio application uses an xml layout file. The XML file acts as a blueprint and tells the tablet where to place certain GUI objects. This XML file is cited in the appendix section of the report.

The XML file for the test application uses the ConstraintLayout container, which allows for flexible positioning and alignment of UI components relative to each other and the parent container. The layout includes three primary UI elements: a Button, a TextView, and an EditText.

The Button has an ID of sendButton which displays text defined by the @string/send resource. The TextView, identified by the ID textView, is initially set as blank and intended for displaying text dynamically during runtime. Lastly, the EditText with the ID sendText serves as an input field where the user can type text. It displays a hint from the @string/input resource to guide the user. All these components are positioned relative to the parent layout using constraints (app:layout_constraintStart_toStartOf and app:layout_constraintTop_toTopOf), ensuring that the layout is in response to the tablet's orientation.

4. Requirements and/or Problems

4.1 Requirements

Our project requirements include designing a user-friendly interface that enables users to schedule deliveries, receive deliveries, and dynamically display the robot's expressions based on its current state or the success criteria of a delivery. It is important that our solution fits neatly into the existing software and hardware that was worked on by previous groups. By the beginning of the next term, we should be able to deliver an interface solution that is largely plug-and-play.

4.2 Constraints

One of the primary challenges we encountered was the dependency on another capstone team responsible for the robot's functionality. Since their work is also scheduled to conclude in April, we had to make several assumptions about the robot's final state while developing our interface. Consequently, testing thus far has been conducted using an ESP32 microcontroller to emulate the robot's behavior, rather than directly interfacing with the robot itself.

Another significant constraint was the lack of prior experience within the team in Android app development. Identifying the appropriate software frameworks and tools that aligned with our requirements needed additional time. Furthermore, setting up these tools with the necessary dependencies and learning to effectively use them was a considerable learning curve.

In terms of computational resources, we were constrained to design our software to not tax the ESP32’s processing capabilities too much, since it is already being used to send commands from the robot and read and interpret signals from the robot. We designed our software to run efficiently, and offloaded what we could onto the Android tablet.

We are also limited by the power supply of the robot. To avoid making things more complicated than necessary, we are relying on the battery of the Android tablet to power our application, so that we avoid dependencies on the power system of R1D4.

5. Design and Results

5.1 Decisions Made

5.1.1 ESP32 Code Fork

We decided to create our own custom fork of the ESP32 code that had previously been written for the R1D4 robot. The rationale behind this decision involved the fact that we want to make our application somewhat modular - It is unclear what future groups will do with the robot, and so we want our

application to be of a “plug and play” nature. Over time, we decided that it was reasonable to assume the correct functionality of the robot commands, as this is not part of the scope of our project.

5.1.2 Android App vs Web App

To develop the interface, we considered two options: a web-based interface and an Android application-based interface (refer to Appendix anjali-A for details). Initially, we planned to implement a web-based interface; however, upon consulting with our advisor we were advised to reconsider based on computational resource constraints. We concluded that developing an Android application to run locally on the tablet would be more efficient since it allows for faster communication with the robot and provides greater access to the tablet's local resources while also being more reliable than a web-based app.

5.1.3 Android tablet Purchase

When choosing which Android tablet to buy, we had a number of considerations. First and foremost, the tablet needed to be affordable since the budget for the R1D4 project is relatively small. After discussing with our supervising professor, we settled on a budget of $80. Within this budget, options on amazon existed but were sparse. In the end, we decided on an Android tablet made by BYYBUO, since it had decent customer reviews and, unlike other tablets in the price range, had 4 GB of ram instead of the typical 2 GB. We decided that this was important because it is difficult to know in advance the resource usage of our frontend.

5.1.4 Frontend Framework

To build the application, we considered various software frameworks, libraries, and tools. Ultimately, we selected React Native with Expo due to its advantages. React Native utilizes JavaScript, a language familiar to some team members, and its beginner-friendly nature facilitated onboarding for the entire team. Additionally, the framework is well-documented, with an extensive range of online resources and guides, which proved invaluable for troubleshooting and learning.

5.2 Results and Progress

5.2.1 Frontend Prototype

Before coding, we designed a Figma prototype of the application to visualize its appearance and functionality. This step allowed us to plan the user interface (UI) comprehensively, ensuring consistency in design elements, identifying the required pages, and finalizing layouts and color schemes. The resulting design aimed to create a dynamic, user-friendly UI.

The application begins with a homepage as seen in figure 2, featuring an animated face that visually represents the robot's emotions. These emotions are dynamically displayed based on the robot's current state or the outcome of its tasks. For example, the face shows a "Happy" expression if a delivery is successfully completed or a "Sad/Upset" expression if the delivery fails due to an error or a “Thinking” expression when it is computing or performing a task.
The homepage of the app has been configured with animations using the Lottie library, which employs JSON-formatted files integrated into the project. These animations dynamically represent the robot’s different phases or states and can switch seamlessly based on the data received from the robot. This enhances the app’s visual appeal while providing an intuitive interface for monitoring the robot’s status.



Figure 2: The implemented Homepage on the Android emulator, where the expression is animated.

The homepage also includes three buttons, each leading to a distinct page, as outlined in the Figma designs:

1. **Send Delivery**: This page presents a form enabling users to schedule a delivery to a specified location. The form allows input of all relevant details, including the delivery location, item name, quantity, time, and an optional image of the item to be delivered.
2. **Manage Tasks**: This page displays a list of all scheduled deliveries that are yet to be completed. It provides users with the necessary information about each delivery and includes options to delete or modify these tasks.
3. **Confirm Pickup**: This page filters deliveries by location, allowing users to view items scheduled for pickup at a specific site. Users can confirm the pickup of an item, marking the delivery as completed within the system.

This structured navigation ensures a seamless and user-friendly experience while effectively managing the robot's tasks.

5.2.2 Control Flow

The control flow of the application, detailing the process of scheduling a delivery and how the app transitions between different pages, is illustrated in figure 3. This diagram provides a comprehensive overview of the user interactions and the corresponding app responses, ensuring a clear understanding of the system's functionality. It highlights the sequential flow of actions, from initiating a delivery request to navigating through various interface pages, such as viewing scheduled tasks, confirming pickups, and returning to the homepage.

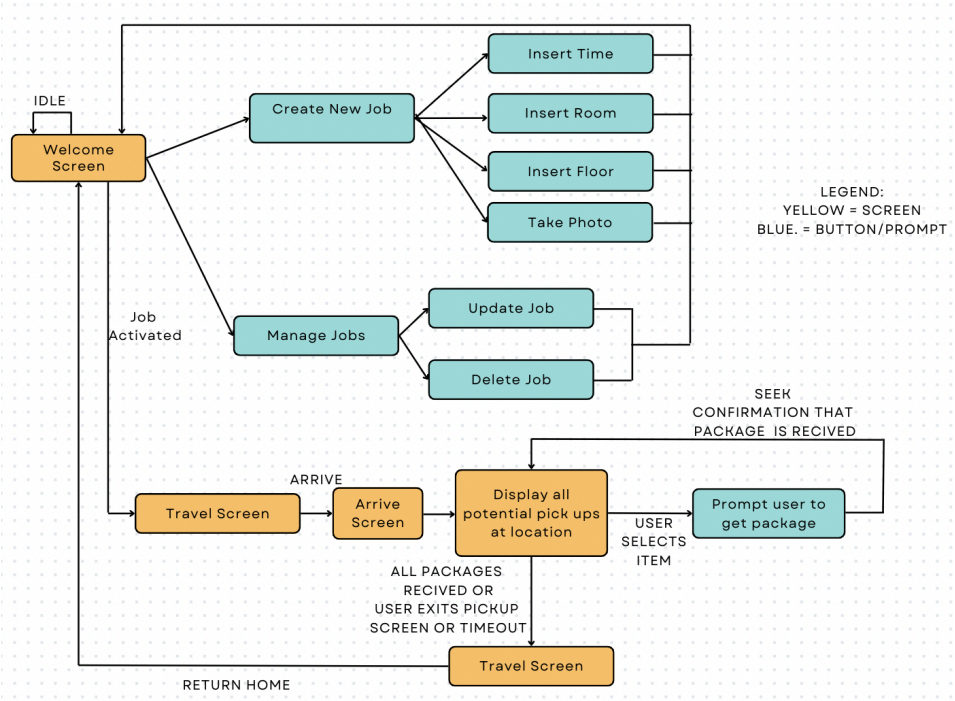


Figure 3: Control flow diagram for the App and shows the logic flow between the different pages.

5.2.3 ESP32 Serial Communication Code

The first step is to include the relevant libraries. We opted to use the stopwatch library for timing operations, SPI for Ethernet communication, and Ethernet.h for handling network interfacing. Then, we establish global variables and constraints. We first run a setup function that initializes the sensor and motor pins, before initializing the serial communication. Then, in the main loop, it continually prints the robot's current state in the serial monitor. It is set up to expect command inputs over the serial port, and when a command is inputted, the code sends it to ProcessCommand for execution. The following are some key functions in the code:

- **ProcessCommand:** This function decodes commands and sets the robot's actions like reading sensors (RFS), stopping motors (SM), moving forward (MF), moving backward (MB), spinning clockwise (SC), and spinning counter-clockwise (SCO).
- **ReadFromSensors:** This function extracts and processes sensor data based on a command.
- **GetDistance:** Calculates the distance to an object using sonar, considering sensor and environmental parameters.
- **EchoDetected:** An ISR to capture the time an echo is detected, aiding in distance measurement.
- **GetCommandFromClient:** Reads and builds a command string from the client via Ethernet.
- In addition, there are motor control functions like **`stopMotor`, `moveForward`, `moveBackward`, `spinClock`**, and **`spinCounter`**, which manage the motor states based on received commands.

5.2.4 ESP32 Telnet Communication Code

Overall, the Telnet code is relatively similar to the serial code. Changes are made only related to using Telnet instead of a wired connection. Instead of including the libraries for wired communication, we include WiFi.h, WiFiClient.h, and WiFiAP.h. A WiFi server is set up with WiFiServer server(23). Later

on, WiFi credentials (ssid and password) are set up to connect to an existing network. The connection to the network and the server startup are managed via WiFi commands (`WiFi.begin(...)`, `WiFi.status()`, etc.).

5.2.5 ESP32 to Tablet Communication

In regards to the data transmitted from tablet to ESP32 to serial, we have developed an application that fully facilities this behavior. We are able to demonstrate that a user operating on the Telnet server can send commands directly to the ESP32 which then the tablet can pick up and display using a test application we made. In this same application, we can show that the tablet can send data to the ESP32 and that the telnet server only picks up certain types of information from the ESP32 and displays this info for the user on the telnet server. For now, we are able to send instructions like “move forward” and “move backwards” to the tablet while the tablet can send a “schedule” file to the telnet server, all through the ESP32. In the test application, a string with the text “schedule” is used to represent the schedule file.

In the final version the following information will be able to be sent from the telnet server and trigger the following actions on the tablet.

- Moving status on telnet → Will trigger the thinking face on tablet
- Arrival status → Will trigger arrived face on tablet
- Error status → Will trigger error face on tablet
- Location at home status Will trigger welcome screen on tablet

Also in the final version, the following information will be able to be sent from the tablet to the telnet server.

- Schedule update on tablet → new schedule on telnet server
- Robot finishing dropping of Packages → notice on telnet server.

For now, we use the test application described in figure 4 below to demonstrate that the communication process is done, even if the full implementation of the app is not working.



Fig. 4: Test Application GUI

When speaking about the communication network between the tablet and the USB, it’s helpful to know what type of problem we are addressing. We know that one of the markers for success in our project is establishing a proper communication channel from the Tablet to the ESP32 and then from the ESP32 to the telnet server. Because the telnet server is a holdover from previous groups, we are constrained to

implement a communication network that does not conflict with this already existing structure. For that reason, we are forced into either a direct connection over Wi-Fi, Bluetooth or wired connection.

Since we are developing a robot that is going to be moving throughout a hospital, it is very possible that at certain points this robot will lose connection with the Wi-Fi in the hospital. For example, when devices enter an elevator, a lot of the time they lose Wi-Fi connection. It is for this reason we elected not to go with a Wi-Fi connection between the tablet and the ESP32. We are now left with Bluetooth versus a physical Connection. After discussions with our project advisor, we deemed it made more sense to have a physical connection since a Bluetooth connection would require the tablet to instruct the ESP32 to spin up its own C++ program on command in order to retrieve the required information. Because of this additional constraint, we decided to go with direct serial communication between the tablet and the ESP32. Serial information is also most easily accessible from the tablet's perspective as we can make it a constant stream of data that the tablet picks up and parses through.

Upon launch, the test application initializes its components in the onCreate method. A TextView is used to display incoming data from the ESP32, while an EditText and a Button allow the user to compose and send text messages. These objects are used as GUI objects that allow the user to interact on the tablet.

After the application has launched, it begins to listen for USB device attachments and detachment events using the onNewIntent method. When a USB device is attached, the app registers a broadcast receiver and requests permission to access the device. Upon receiving permission, it calls the connect method to establish the communication link. During detachment, the app sees to it that the USB connection is safely closed, preventing crashes.

The connect method handles the core communication setup. It leverages the UsbSerialProber library to identify connected USB devices and their corresponding drivers. Once a suitable driver is found, the app opens a connection to the device, retrieves its serial port, and configures it with specific parameters: a baud rate of 115200, 8 data bits, 1 stop bit, and no parity. This configuration ensures compatibility with typical ESP32 serial communication settings. After the port is initialized, a background thread is launched to handle incoming data from the ESP32 while the connection remains active.

Incoming data is read from the USB port in the background thread. The test application uses a buffer to collect data in chunks, which is then appended to a StringBuilder. It assumes that the ESP32 terminates each message with a newline character (\n), enabling the app to detect and process complete messages. This mechanism allows the app to fully read the incoming text data . Without this StringBuilder object and the corresponding for loop, the application only grabs the first character from the Serial data stream. This bug was discovered during our testing stage of this application.Once a message is received, it is displayed on the tablet's TextView by updating the UI thread via the runOnUiThread method. This makes sure the UI is updated safely and without blocking the main thread.

To send data to the ESP32, the app writes the contents of the EditText field to the serial port as UTF-8 encoded bytes. Synchronization mechanisms, using a shared object (porting), are employed to maintain thread-safe access to the USB port during both sending and receiving operations. This prevents resource conflicts and ensures reliable communication. The background thread continuously monitors the port's state, terminating gracefully if the port is closed or if the connection is interrupted.

Once the data is received on the ESP32, a basic 'if else' statement is used in the loop section of the code to check if the received string matches the type of a schedule file. If there is a match, the ESP32 displays the data on to the telnet server, specifically the client side. In the case of the test application, matching the schedule file type just means that it's the string "schedule", but this will be changed in further versions of the code to be a check for either a CSV data type or possibly some other check.

This concludes an explanation of how the communication works between the tablet, ESP32 and Telnet server, with an emphasis on how the communication works between the Android tablet and the ESP32.

6. Plan for Next Semester

Next semester, we will finalize the R1D4 touchscreen interface and transition to the Swarm Robots project. Over the winter break, we aim to complete and refine the Android app, integrating it with the ESP32 and Telnet server. Once operational, the R1D4 system will undergo thorough testing before focusing on the swarm robots.

6.1 Remaining Task and Timeline

During the winter break, we will focus on finalizing the development of the R1D4 robot's touchscreen interface. This includes completing the remaining sections of the Android application, so the interface is fully implemented and functional. After finalizing the Android app, we have to make sure that it integrates with the ESP32 microcontroller, and connects to the Telnet server. The app's frontend, built with React Native and Expo, will be completed and tested on an Android emulator before being deployed on the tablet itself. Meanwhile, we aim to refine the backend, which processes user inputs from the app and stores the data locally in a CSV file. The backend will also facilitate communication with the ESP32 to send commands and receive updates. Our goal is for each team member to finalize their part of the work, ensuring the app and its backend are fully functional and reliable.

6.2 Testing Plan

Testing is important to make sure the R1D4 functions as expected. We will begin with unit testing individual components, including the React Native frontend and ESP32 backend. In order to evaluate the usability and responsiveness of the Android app we will be using the Android Studio emulator, which replicates various device configurations and API levels. System testing will validate integration between the app and ESP32, focusing on key functionalities like sending movement commands and receiving confirmations. This will include verifying that the ESP32 accurately processes commands and updates registers. Communication with the Telnet server will also be tested to verify accurate data exchange across components.

Finally, we will simulate real-world usage scenarios, such as task scheduling and manual control, to make sure the interface can function well in a real-world hospital.

6.3 Testing-Oriented Design Decisions

To simplify testing, data handling will remain local and stored in CSV files, at least based on our current plans. However, this approach may change in the future depending on more detailed requirements and feasibility. The app's user interface will include clear, labeled buttons for tasks like scheduling deliveries and confirming pickups.

The ESP32's error-handling capabilities will be utilized to detect and report communication failures. Modular and incremental testing will make sure components work independently and in coordination. Diagnostic tools in the app should provide real-time feedback on communication and errors.

6.4 Project Objectives

Each team member is tasked with completing their assigned responsibilities to make steady progress. Our immediate goal is to deliver a fully functional system before the Mid-Project Presentation, where the Android tablet intracts reliably with the ESP32, managing communication with the robot and the Telnet server. In essence, if we can create an intuitive and robust interface, we deem this to be a success.

Finally, after the Mid-Project Presentation for the R1D4, we plan to shift our focus on the Swarm Robots. This will include optimizing communication through a central Wi-Fi server, integrating hardware upgrades like sensors and cameras, and enabling the robots to autonomously detect and transport objects. Additionally, we hope to develop AI algorithms to manage both individual robot actions and collective swarm behavior.

7. Impact on Society and the Environment

The R1D4 robot's touchscreen interface has significant societal and environmental impacts. While it enhances efficiency in healthcare and other industries, it also raises concerns about resource usage, safety, and sustainability.

7.1 Use of Non-Renewable Resources

Our work on the R1D4 robot focuses on enhancing its touchscreen interface rather than building the robot itself. However, this enhancement involves non-renewable resources, such as the materials used in the tablet, sensors, and microcontroller (ESP32). These components contribute to electronic waste if not properly managed, and the energy required for their production and operation increases the overall carbon footprint. To address these concerns, our design prioritizes durability and modularity, allowing for upgrades and reuse of existing components. While the robot’s reliance on non-renewable resources poses challenges, strategies like modular design and local data storage minimize waste and reduce environmental impact. For instance, instead of replacing the tablet, we made sure to keep the computational requirements light so that the hardware does not need to be upgraded.

7.2 Environmental Benefits

Our project improves hospital operations by automating tasks like medication delivery. Due to the intelligent nature of scheduling deliveries bundled together, the total amount of energy burned per delivery can be reduced. While small, this lowers the overall energy footprint. Additionally, the modular design of the interface allows for regular software upgrades without requiring hardware replacements, significantly cutting down on electronic waste.

7.3 Safety and Risk

During our development, safety risks to our team included potential hardware malfunctions and programming errors that could result in frying circuit boards or unintended robot behavior. As any failure could jeopardize patient care, we need to make sure commands sent to the robot are carried out in a predictable manner. To facilitate this, the interface is designed to be as user-friendly as possible.

7.4 Benefits to Society

The R1D4 robot may provide some societal benefits, particularly in healthcare settings. By automating tasks like medication delivery, it reduces staff workload, allowing them to dedicate more time to patient care. Economically, the robot reduces costs associated with manual labor and operational inefficiencies, making healthcare services more effective. Finally, the robot and its interface offer a

solution to physically disabled healthcare workers that are unable to transport goods from point A to point B.

8. Teamwork Report

8.1 Team Collaboration

Overall, our team collaborated in an organized manner. We made sure to meet at least once per week to discuss progress. During these meetings we collectively made key decisions, such as opting for an app-based approach over a website and selecting a suitable tablet within our budget. To expedite efficiency, the team was divided into two subgroups: Paolo and Joey focused on backend infrastructure and hardware communication, while Anjali and Noorjahan worked on the app's user interface, including design, animations, and integration with React Native. This structure allowed parallel progress and steady advancements across all project areas.

8.2 Individual Contributions

Paolo Bizzari: Paolo's main contributions revolved around establishing communication between the hardware and software components. He worked on getting the old code from the previous group running on the ESP32, setting up serial communication to relay messages between the ESP32 and a laptop, and enabling serial communication between the tablet and ESP32. He also created a test application to demonstrate communication between the Telnet server, tablet, and ESP32 and confirmed that the application worked on the tablet hardware. Additionally, Paolo procured the tablet and designed a control flow diagram detailing the tablet's behavior.

Joseph Hershkop: Joey's contributions focused on bridging the structural aspects of the frontend with the program logic. He contributed to the Figma design and created an architectural diagram outlining the structure of the ESP32, tablet, laptop server, and robot. Together with Paolo, he worked on adapting the old code from the previous group to run on the ESP32. Joey also configured a Wi-Fi network to accept a Telnet server setup and successfully established the Telnet server on the ESP32, enabling the relay of messages to and from a remote terminal session.

Anjali Singhal: Anjali's work focused on developing the app interface. She designed the Figma prototype to plan the user interface and configured the React Native framework with Expo, including adding relevant dependencies. She contributed to both the frontend and backend development of the app and animated the robot's expressive faces using the Lottie library. She took time to find and learn software frameworks and tools for the app.

Noorjahan Abonty: Noorjahan contributed to the project by assisting in the selection and setup of the software framework and tools for the app, including exploring configurations for React Native and Expo. She helped in the preparation of the midterm report, highlighting the pros and cons of implementing a website versus an Android app, which guided our team in making their final decision based on our project advisor's input. While final exams limited her direct work on the app development, she plans to actively contribute in future by developing and testing both the frontend and backend code of the app to ensure seamless integration with the tablet, ESP32, and robot, preparing it for final deployment.

8.3 Challenges and Solutions

Our team worked together effectively, but faced a few challenges with debugging old code, getting familiarized with React Native, and understanding the software interactions between the ESP32, tablet, laptop server, and robot. To address these, our team met with each other frequently to bounce ideas off one-another, and met with our project advisor when we needed guidance.

9. Conclusion

This semester, we made significant progress with the R1D4 robot by establishing a working communication network between the ESP32, Telnet server, and the Android tablet. We also developed a prototype for the Android app, featuring robot facial expressions and basic functionality. Important decisions, like choosing an Android app over a web interface and designing a modular system, provided a strong foundation for further development. In the coming semester, we will focus on completing the app, integrating it with the hardware, and thoroughly testing the system. After completing this phase for R1D4, we plan to transition to the Swarm Robot project, where we aim to communicate to the robots via a central Wi-Fi server, add sensors and data sources to enhance the swarm's view of the world, and develop AI algorithms for managing navigation, object detection, and problem solving tasks.

10. References

[1] M. Mik3y, "USB Serial for Android," GitHub, 2024 [Online]. Available: https://github.com/mik3y/usb-serial-for-Android/tree/master/tab=readme-ov-file. [Accessed: Dec. 18, 2024].

[2] M. Sulkiewicz, "What Is React Native? Complex Guide for 2024 – Netguru," Netguru, 2024 [Online]. Available: https://www.netguru.com/glossary/react-native [Accessed: Dec. 18, 2024]

[3] React Native. "Get Started with React Native." reactnative.dev, [Online]. Available: https://reactnative.dev/docs/environment-setup [Accessed: Dec. 18, 2024]

11. Appendices

Fig. A1: Test Application Java Code

```
package com.example.testusbcommunication;
import Android.annotation.SuppressLint;
import Android.app.PendingIntent;
import Android.content.BroadcastReceiver;
import Android.content.Context;
import Android.content.Intent;
import Android.content.IntentFilter;
import Android.hardware.usb.UsbDevice;
import Android.hardware.usb.UsbDeviceConnection;
import Android.hardware.usb.UsbManager;
import Android.os.Bundle;
import Android.view.View;
import Android.widget.Button;
import Android.widget.EditText;
import Android.widget.TextView;
import Androidx.appcompat.app.AppCompatActivity;
import com.hoho.Android.usbserial.driver.UsbSerialDriver;
import com.hoho.Android.usbserial.driver.UsbSerialPort;
import com.hoho.Android.usbserial.driver.UsbSerialProber;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Map;
/**
This is a base example of this project. It holds code for basic communication between esp microcontroller and
Android app.
*/
public class MainActivity extends AppCompatActivity {
  private TextView textView;
  private EditText sendText;
  private Button sendButton;
  UsbDeviceConnection connection;
  List<UsbSerialDriver> availableDrivers;
  UsbSerialDriver driver;
  UsbSerialPort port;
  UsbManager manager;
  Map<String, UsbDevice> devices;
  UsbDevice device = null;
  PendingIntent pi;
  private boolean running= false;
  private final Object porting =new Object();
  private static String ACTION_USB_PERMISSON = "com.Android.example.USB_PERMISSION";
  private final BroadcastReceiver receiver = new BroadcastReceiver(){
    @Override
    public void onReceive(Context context, Intent intent){
      String action = intent.getAction();
      if(ACTION_USB_PERMISSON.equals(action))
        synchronized (this){
        if(manager.hasPermission(device))
          connect();
```

```
            }
        }
    };
    private void connect(){
        availableDrivers = UsbSerialProber.getDefaultProber().findAllDrivers(manager);
        driver = availableDrivers.get(0);
        connection = manager.openDevice(driver.getDevice());
        port = driver.getPorts().get(0);
        try {
            port.open(connection);
            port.setParameters(115200,8,UsbSerialPort.STOPBITS_1,UsbSerialPort.PARITY_NONE);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        new Thread(new Runnable() {
            @Override
            public void run() {
                running = true;
                while(running){
                    synchronized (porting){
                        if(!port.isOpen())
                            break;
                    }
                    byte[] data = new byte [500];
                    synchronized (porting){
                        try {
                            byte[] buffer = new byte[1024];
                            StringBuilder incomingMessage = new StringBuilder();
                            int numBytesRead;
                            while ((numBytesRead = port.read(buffer, 2000)) > 0) {
                                String receivedData = new String(buffer, 0, numBytesRead, StandardCharsets.UTF_8);
                                incomingMessage.append(receivedData);
                                if (receivedData.contains("\n")) {   break;
                                }
                            }
                            runOnUiThread(new Runnable() {
                                @Override
                                public void run() {
                                    textView.setText("");
                                    textView.setText(incomingMessage.toString().trim());
                                }
                            });
                            Thread.sleep(200);
                            synchronized (porting){
                                running = port.isOpen();
                            }
                        } catch (IOException | InterruptedException e) {
                            throw new RuntimeException(e);
                        }
                    }
                }
            }
        }).start();
    }
    @Override
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        textView = findViewById(R.id.textView);
        sendButton = findViewById(R.id.sendButton);
        sendText = findViewById(R.id.sendText);

        manager = (UsbManager) getApplicationContext().getSystemService(Context.USB_SERVICE);
        devices = manager.getDeviceList();
        device = null;
        sendButton.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View v){
                try {
                    synchronized (porting){
                        port.write(sendText.getText().toString().getBytes(StandardCharsets.UTF_8),2000);
                        sendText.setText("");
                    }
                } catch (IOException e) {
                    throw new RuntimeException(e);
                }
            }
        });
    }
```

```
  @SuppressLint("UnspecifiedRegisterReceiverFlag")
  @Override
  protected void onNewIntent(Intent intent) {
    super.onNewIntent(intent);
    if(intent.getAction().equals(UsbManager.ACTION_USB_DEVICE_ATTACHED)){
      IntentFilter filter = new IntentFilter(ACTION_USB_PERMISSON);
      registerReceiver(receiver,filter);
      for(Map.Entry<String, UsbDevice>entry:devices.entrySet())
        device = entry.getValue();
      pi = PendingIntent.getBroadcast(getApplicationContext(),0,
          new Intent(ACTION_USB_PERMISSON),PendingIntent.FLAG_IMMUTABLE);
      manager.requestPermission(device,pi);
    }else if(intent.getAction().equals(UsbManager.ACTION_USB_DEVICE_DETACHED)){
      synchronized (porting){
        try {
          port.close();
          connection.close();
        } catch (IOException e) {
          throw new RuntimeException(e);
        }
      }
    }
  }
}
```

Fig. A2: ESP32 Code from past group + our additions to facilitate tablet communication

```
#include "StopWatch.h"
// Include the necessary libraries
#include <WiFi.h>
#include <WiFiClient.h>
#include <WiFiAP.h>
// GLOBAL VARIABLES
int SensorPins[] = {31, 33, 35, 37};  // The Sonar Ranging Module accepts sensor nbr in binary format only. LSB = 31
and MSB = 37. Put them in an array for easy access in code
const int Init = 47, Echo = 49, Binh = 51;  // Link pins 30, 44 & 46 to "Initialize", "Echo", and "BINH" signals
respectively
StopWatch SW(StopWatch::MICROS); // StopWatch to measure time (in us)
volatile float TimeToEcho = 0; // This variable will be used to pass time values between ISR and code (in us)
// GLOBAL CONSTANTS FOR MOTOR CONTROL
const int motorPin1 = 2;
const int motorPin2 = 22;
const int motorPin3 = 4;
const int motorPin4 = 12;
const int motorPin5 = 14;
const int motorPin6 = 16; // Changed from 7
const int motorPin7 = 17; // Changed from 8
const int motorPin8 = 18; // Changed from 9
String state = "";
// WiFi Configuration
const char* ssid = "Paolosiphone";
const char* password = "12345678";
// SERVER SETUP
WiFiServer server(23); // telnet defaults to port 23
// COMMUNICATION MODE FLAG
bool useTelnet = true; // Set this to false if you want to use Serial instead
void setup() {
  Serial.begin(115200);
  // Initialize pins
  for (int i = 0; i < 4; i++) { pinMode(SensorPins[i], OUTPUT); }
  pinMode(Init, OUTPUT);
  pinMode(Echo, INPUT);
  pinMode(Binh, OUTPUT);
  attachInterrupt(digitalPinToInterrupt(Echo), EchoDetected, RISING);
  // Motor control pins initialization
  analogWrite(motorPin1, OUTPUT);
  analogWrite(motorPin2, OUTPUT);
  analogWrite(motorPin3, OUTPUT);
  analogWrite(motorPin4, OUTPUT);
  analogWrite(motorPin5, OUTPUT);
  analogWrite(motorPin6, OUTPUT);
  analogWrite(motorPin7, OUTPUT);
  analogWrite(motorPin8, OUTPUT);
  if (useTelnet) {
    // Connect to WiFi
```

```
    Serial.println("Connecting to WiFi...");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
      delay(1000);
      Serial.println("Connecting...");
    }
    Serial.println("Connected to WiFi.");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
    server.begin();
    Serial.println("Telnet server started.");
  } else {
    Serial.println("Serial communication started. Please enter command here.");
  }
}
// MAIN LOOP
void loop() {
  //Serial.println(state);
  //delay(500);
  if (useTelnet) {
    // Handle Telnet Client
    WiFiClient client = server.available();
    if (client) {
      Serial.println("Client connected.");
      client.println("Hello, client! Please enter a command.");
      while (client.connected()) {
        Serial.println(state);
        delay(500);
        if (client.available()) {
          String ClientCommand = GetCommandFromClient(client);
          if(ClientCommand.equals("end")){
            client.stop();
          }
          ProcessCommand(ClientCommand, client);
        }else{
          if(Serial.available() > 0) {
            String serialLine = Serial.readStringUntil('\n');
            while (Serial.available() > 0) {
                Serial.read(); // Clear buffer before reading fresh data
            }
            if(serialLine.equals("schedule")){
              //Serial.println("Schedule Received!!");
              client.print("Schedule Received!!");
              delay(5000);
            }
        }
      }
      }
      client.stop();
      Serial.println("Client disconnected.");
  }
  } else {
    // Handle Serial Communication
    if (Serial.available() > 0) {
      Serial.println("Serial data detected!");
      String SerialCommand = Serial.readStringUntil('\n');
      SerialCommand.trim();
      if (SerialCommand.length() > 0) {
        Serial.print("Command received: ");
        Serial.println(SerialCommand);
        Serial.println("Processing command...");
        ProcessCommand(SerialCommand, Serial);
      } else {
        Serial.println("Received empty command.");
      }
      if (Serial.available() > 0) {
        Serial.println("Warning: Data still in buffer!");
      } else {
        Serial.println("Buffer empty as expected.");
      }
      Serial.println("Waiting for the next command...");
    } else {
      Serial.println("No serial data available.");
      delay(1000); // Avoid spamming the output
    }
  }
}
```

```
// FUNCTION TO SELECT COMMAND PROCESSING
void ProcessCommand(String command, Print &output) {
    if (command.indexOf("RFS") != -1) {
        //output.println(ReadFromSensors(command));
        state = "Read From Sensors";
    } else if (command.indexOf("SM") != -1) {
        //Serial.println("Stop Motor");
        state = "Stop Motors";
        stopMotor();
    } else {
        String argument = command.substring(command.indexOf(" ") + 1);
        argument.replace(" ", "");
        if (command.indexOf("MF") != -1) {
            moveForward(argument.toInt());
            state = "Moving Forward ";
            state.concat(argument.toInt());
            state.concat(" Meters");
        } else if (command.indexOf("MB") != -1) {
            moveBackward(argument.toInt());
            state = "Moving Backward ";
            state.concat(argument.toInt());
            state.concat(" Meters");
        } else if (command.indexOf("SCO") != -1) {
            spinCounter(argument.toInt());
            state = "Spin Counter Clockwise";
            state.concat(argument.toInt());
            state.concat(" Degrees");
        } else if (command.indexOf("SC") != -1) {
            spinClock(argument.toInt());
            state = "Spin Clockwise";
            state.concat(argument.toInt());
            state.concat(" Degrees");
        }
    }
}
// MAIN FUNCTIONS
String ReadFromSensors(String ClientCommand){
  // Decrypt Client Command and get the number of sensors to be echoed
  ClientCommand = ClientCommand.substring(ClientCommand.indexOf("<") + 1);
  ClientCommand = ClientCommand.substring(0, ClientCommand.indexOf(">"));
  ClientCommand.replace(" ", "");
  int SeperatorAtIndex = ClientCommand.indexOf(",");
  int NbrOfSeparators = 0;
  while (SeperatorAtIndex != -1) {
    NbrOfSeparators = NbrOfSeparators + 1;
    SeperatorAtIndex = ClientCommand.indexOf(",", SeperatorAtIndex + 1);
  }
  // Read sensor numbers and then read distance from each one of them and send back to client
  int SensorNbrs[NbrOfSeparators + 1];
  float SensorReadings[NbrOfSeparators + 1];
  String Output = "";
  if (NbrOfSeparators == 0){
    SensorNbrs[0] = ClientCommand.toInt();
    SensorReadings[0] = GetDistance(SensorNbrs[0]);
    Output = (String)SensorReadings[0];
  } else {
    for (int i = 0; i < NbrOfSeparators + 1; i++){
      SensorNbrs[i] = ClientCommand.substring(0, ClientCommand.indexOf(",")).toInt();
      SensorReadings[i] = GetDistance(SensorNbrs[i]);
      Output = Output + (String)SensorReadings[i] + " ";
      ClientCommand = ClientCommand.substring(ClientCommand.indexOf(",") + 1);
    }
  }
  return Output;
}
// HELPER FUNCTIONS
float GetDistance(int SensorNbr){
  // Decode the sensor nbr from decimal (0 to 15) to binary, pass it to corresponding pins on the board, then read
distance from that sensor and return it (in meters)
  for (int i = 0; i < 4; i++){
    if (((SensorNbr/(int)pow(2, i)) % 2) == 1){digitalWrite(SensorPins[i], HIGH);} else {digitalWrite(SensorPins[i],
LOW);}
  }
  // Declare needed variables
  const float BinhTime = 0.65; // BINH rise time (in ms). This signal reduces the blanking period
  const float SpeedOfSound = 340.29; // Speed of sound (in m/s)
  const float MaxDistance = 5; // Max wanted distance to be echoed by sensor (in meters). The max recommended value is
```

```
10m (for accuracy reasons)
  const float MaxHearingTime = 2*MaxDistance/SpeedOfSound; // This is the maximum "hearing" time allowed (in seconds)
  float Distance = 0;
  // Calculate distance between sensor and obstacle (in meters) and return the value
  digitalWrite(Init, LOW); digitalWrite(Binh, LOW);
  delay(1); // Reset the interface
  SW.reset(); TimeToEcho = 0; digitalWrite(Init, HIGH); SW.start();// Bring Init signal to high and start the timer
  do {} while (SW.value() < BinhTime*1000); digitalWrite(Binh, HIGH); // Bring BINH sinal to high after BinhTime to
reduce Blanking time -> read shorter distances
  do {} while (SW.value() < MaxHearingTime*1000000); // Wait for rising edge on Echo signal
  Distance = SpeedOfSound*TimeToEcho/2000000;
  return Distance;
}
void EchoDetected(){
  // ISR for RISING edge on Echo signal, this detects when the sound wave gets back to the sensor and saves
corresponding stopwatch value
  TimeToEcho = SW.value();
}
String GetCommandFromClient(WiFiClient client){
  // As long as the client hasn't pressed the 'Enter' key, the 'ClientCommand' string will keep building
  String ClientCommand = "";
  char IncomingChar = (char)client.read();
  while (IncomingChar != '\n'){
    if (IncomingChar != (char)-1) {ClientCommand = ClientCommand + (String)IncomingChar;}
    IncomingChar = (char)client.read();
  }
  ClientCommand.trim();
  return ClientCommand;
}
// MOTOR CONTROL FUNCTIONS
void stopMotor(){
  analogWrite(motorPin1, 0);
  analogWrite(motorPin2, 0);
  analogWrite(motorPin3, 0);
  analogWrite(motorPin4, 0);
  analogWrite(motorPin5, 0);
  analogWrite(motorPin6, 0);
  analogWrite(motorPin7, 0);
  analogWrite(motorPin8, 0);
}
void moveForward(int distance){
  // we know from testing that robot speed is 36 cm/s
  //Serial.println("Moving Forward Function");
  analogWrite(motorPin1, 100);
  analogWrite(motorPin2, 100);
  analogWrite(motorPin3, 100);
  analogWrite(motorPin4, 0);
  analogWrite(motorPin5, 100);
  analogWrite(motorPin6, 100);
  analogWrite(motorPin7, 100);
  analogWrite(motorPin8, 100);
  if (distance != 0) {
    int delayTime; delayTime = 1000 * distance / 36;
    delay(delayTime);
    stopMotor();
  }
}
void moveBackward(int distance){
  // we know from testing that robot speed is 36 cm/s
  analogWrite(motorPin1, 100);
  analogWrite(motorPin2, 0);
  analogWrite(motorPin3, 100);
  analogWrite(motorPin4, 100);
  analogWrite(motorPin5, 100);
  analogWrite(motorPin6, 100);
  analogWrite(motorPin7, 100);
  analogWrite(motorPin8, 100);
  if (distance != 0) {
    int delayTime; delayTime = 1 * distance / 36;
    delay(delayTime); stopMotor();
  }
}
void spinClock(int angle){
  // we know from testing that a 90deg spin takes about 700ms
  int delayTime;
  delayTime = angle * 700 / 90;
  analogWrite(motorPin1, 100);
```

```
  analogWrite(motorPin2, 100);
  analogWrite(motorPin3, 100);
  analogWrite(motorPin4, 100);
  analogWrite(motorPin5, 100);
  analogWrite(motorPin6, 100);
  analogWrite(motorPin7, 100);
  analogWrite(motorPin8, 100);
  delay(delayTime); stopMotor();
}
void spinCounter(int angle){
  // we know from testing that a 90deg spin takes about 700ms
  int delayTime;
  delayTime = angle * 700 / 90;
  analogWrite(motorPin1, 100);
  analogWrite(motorPin2, 0);
  analogWrite(motorPin3, 100);
  analogWrite(motorPin4, 0);
  analogWrite(motorPin5, 0);
  analogWrite(motorPin6, 0);
  analogWrite(motorPin7, 0);
  analogWrite(motorPin8, 0);
  delay(delayTime); stopMotor();
}
```

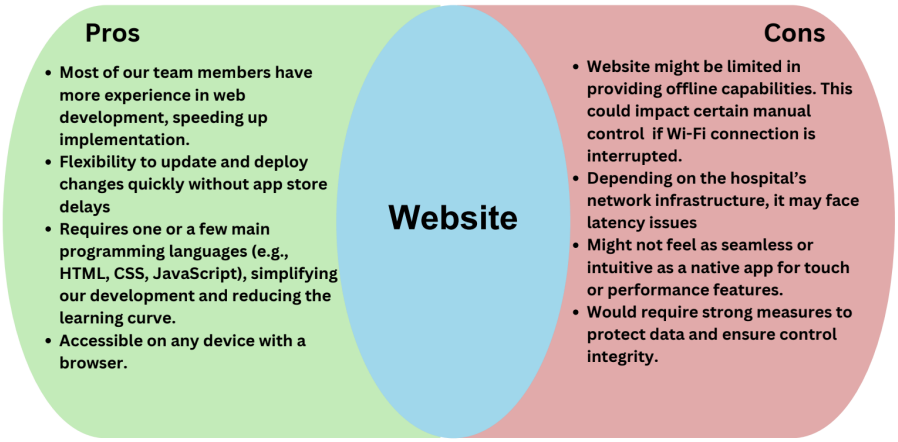


Fig. A3: List of the pros and cons of implementing a website.

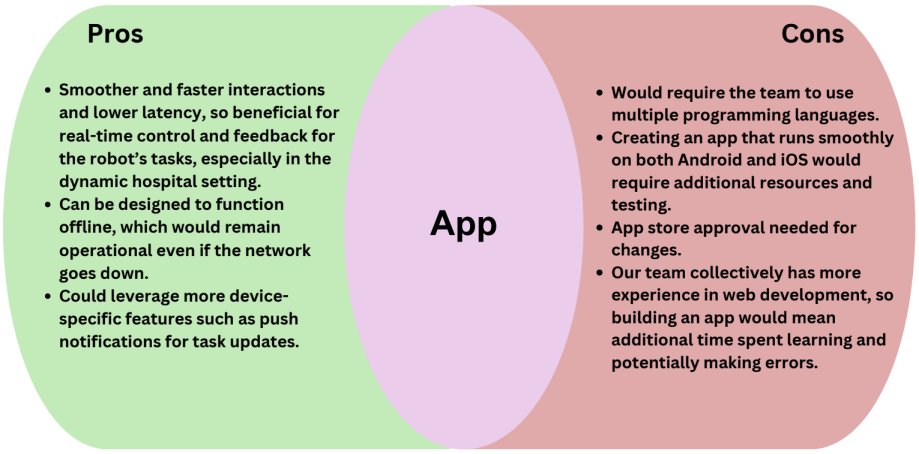


Fig. A4: List of the pros and cons of implementing an Android application..